

Pula Plataformas

Relatório de Implementação

Trabalho Final — Introdução aos Sistemas Embarcados

Bruno Suxberger Valadares Araújo — matrícula 231012728

Pedro Araujo Cordeiro Viana — matrícula 202067452

15 de julho de 2026

Resumo

Este relatório descreve a implementação do *Pula Plataformas*, jogo digital desenvolvido para a placa DE1-SoC como trabalho final da disciplina de Introdução aos Sistemas Embarcados. O jogo é um *platformer* vertical (estilo *Doodle Jump*) em que o personagem sobe pulando entre plataformas de gelo geradas proceduralmente, desviando de um vilão-nave que patrulha o topo da tela e atira projéteis que podem ser refletidos de volta por meio de uma mecânica de *parry*. O documento cobre como compilar e executar o código, o funcionamento do jogo, as entradas e saídas utilizadas, a estrutura do código modelada em UML, o pipeline gráfico, o sistema de pontuação e recordes, e as principais dificuldades encontradas durante o desenvolvimento.

Sumário

1	Introdução	2
2	Como compilar e executar	2
2.1	Simulação do CPUlator ao MSYS2	2
2.2	Compilando com MSYS2 (SDL2) — fluxo usado no desenvolvimento	3
2.3	Compilando para a DE1-SoC real	3
2.4	Execução na placa DE1-SoC	4
3	Funcionamento do jogo	5
4	Entradas e saídas	7
5	Estrutura do código (UML)	8
5.1	Diagrama de estados do vilão	9
5.2	Diagrama de sequência: parry e reflexo do projétil	9
5.3	Principais módulos do arquivo	10
6	Gráficos	10
6.1	Framebuffer e <i>double buffering</i>	10
6.2	Pipeline de sprites	10
6.3	Fundo e câmera	11

7	Sistema de pontuação	11
7.1	Pontuação	11
7.2	Recordes persistentes	11
8	Dificuldades e desafios	12
9	Conclusão	13

1 Introdução

O trabalho consiste em um jogo de plataforma para a DE1-SoC, utilizando o vídeo VGA, os mostradores de 7 segmentos e os periféricos de entrada da placa (chaves e botões). Diferente do jogo de deslocamento horizontal estilo “Mário” sugerido no enunciado original, optou-se por um jogo de escalada vertical infinita: o personagem (Kirby) sobe pulando de plataforma em plataforma, tentando alcançar uma pontuação máxima antes que suas vidas acabem, enquanto enfrenta um vilão que atira nele do topo da tela.

O código-fonte principal está em `pula_plat.c` e é organizado em torno de um laço de jogo clássico (leitura de entrada, atualização de estado, renderização), com suporte a três alvos de compilação diferentes por meio de *flags* do pré-processador, descritos na Seção 2.

2 Como compilar e executar

O jogo foi escrito para compilar em três ambientes diferentes, selecionados por *flags* de compilação, sem alterar a lógica do jogo:

- **DE1-SoC real** (`-DDE1_SOC`): acessa os endereços de memória da VGA, dos LEDs, das chaves e dos botões diretamente via `/dev/mem`, exigindo execução como *root*.
- **CPUlator**, sem nenhuma *flag*: os mesmos endereços de memória usados na DE1-SoC real são acessíveis diretamente dentro do simulador, sem necessidade de `mmap`.
- **SDL_SIM** (`-DSDL_SIM`): substitui a VGA por uma janela SDL2 e as chaves/botões pelo teclado, permitindo testar o jogo em um PC comum (Windows, via MSYS2), sem precisar da placa nem do simulador.

2.1 Simulação do CPUlator ao MSYS2

No início do desenvolvimento, o projeto foi testado exclusivamente no **CPUlator** (<https://cpulator.01xz.net>), um simulador da DE1-SoC que roda inteiramente no navegador. Ele é útil por não exigir acesso à placa física, mas impõe limitações sérias para um projeto que cresce em complexidade: aceita apenas um único arquivo `.c` colado no editor (sem suporte a múltiplos arquivos ou `#include` de arquivos locais), não possui um sistema de arquivos persistente entre execuções (impossibilitando, por exemplo, salvar o arquivo de recordes descrito na Seção 7), e o ciclo de teste — editar, colar no navegador, rodar — é sensivelmente mais lento do que compilar e rodar localmente.

À medida que o projeto ganhou sprites (múltiplos arquivos `.h` gerados separadamente), um vilão, um sistema de recordes persistente e um laço de testes muito mais iterativo, tornou-se preferível desenvolver e testar localmente usando o **MSYS2**. MSYS2 é uma

distribuição de software para Windows que fornece um ambiente do tipo Unix (terminal, utilitários *shell*) e um gerenciador de pacotes (**pacman**) com *toolchains* pré-compiladas — em particular o MinGW-w64, uma versão do GCC que gera executáveis nativos do Windows (**.exe**) em vez de exigir uma máquina virtual Linux ou o WSL. Combinado com a biblioteca SDL2 (que abstrai janela, teclado e *framebuffer*), isso permite compilar e rodar o jogo localmente com um laço de desenvolvimento rápido, mantendo o mesmo código C que roda na placa.

O arquivo `pula_plat.c` usa `#include` para os *headers* de sprite (ver Seção 6), o que o CPULATOR não aceita. Por isso, o repositório do projeto mantém um script auxiliar, `tools/build_cpulator.py`, que gera uma versão de `pula_plat.c` com todos os `#include` locais “inlinados” em um único arquivo (`pula_plat_cpulator.c`), apenas para os casos em que ainda seja necessário validar no simulador. Esse arquivo é puramente derivado — não é editado diretamente, e é regenerado a partir de `pula_plat.c` sempre que necessário; o desenvolvimento e os testes do dia a dia acontecem no MSYS2.

2.2 Compilando com MSYS2 (SDL2) — fluxo usado no desenvolvimento

1. Instalar o **MSYS2** e abrir o terminal “**MSYS2 MinGW 64-bit**” (não o “MSYS2 MSYS” genérico, que não gera binários nativos do Windows).
2. Instalar o compilador e a SDL2:

```
pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-sdl2
```

3. Compilar:

```
gcc -DSDL_SIM pula_plat.c -o jogo.exe -lSDL2 -lm
```

4. Executar (de dentro do mesmo terminal, que já tem o `SDL2.dll` no `PATH`):

```
./jogo.exe
```

Para rodar fora desse terminal, basta copiar `SDL2.dll` para a mesma pasta do executável.

Os controles no modo SDL são o teclado: setas ou WASD para movimento horizontal, espaço ou seta para cima para pular, e *shift* para o *parry*. Pontuação e vidas aparecem no título da janela, já que não há mostrador de 7 segmentos nem LEDs de verdade nesse modo.

2.3 Compilando para a DE1-SoC real

```
gcc -DDE1_S0C pula_plat.c -o jogo -lm
sudo ./jogo      # precisa de root para acessar /dev/mem
```

2.4 Execução na placa DE1-SoC

Para validar o funcionamento fora do CPULator, o jogo também foi executado diretamente na placa DE1-SoC real, com o VGA da placa exibindo a tela do jogo e os LEDs/HEX refletindo o estado das vidas e da pontuação (ver `atualiza_leds` e `atualiza_display`). A Figura 2 mostra a placa em funcionamento em diferentes momentos da partida.



Figura 1: Tela de início

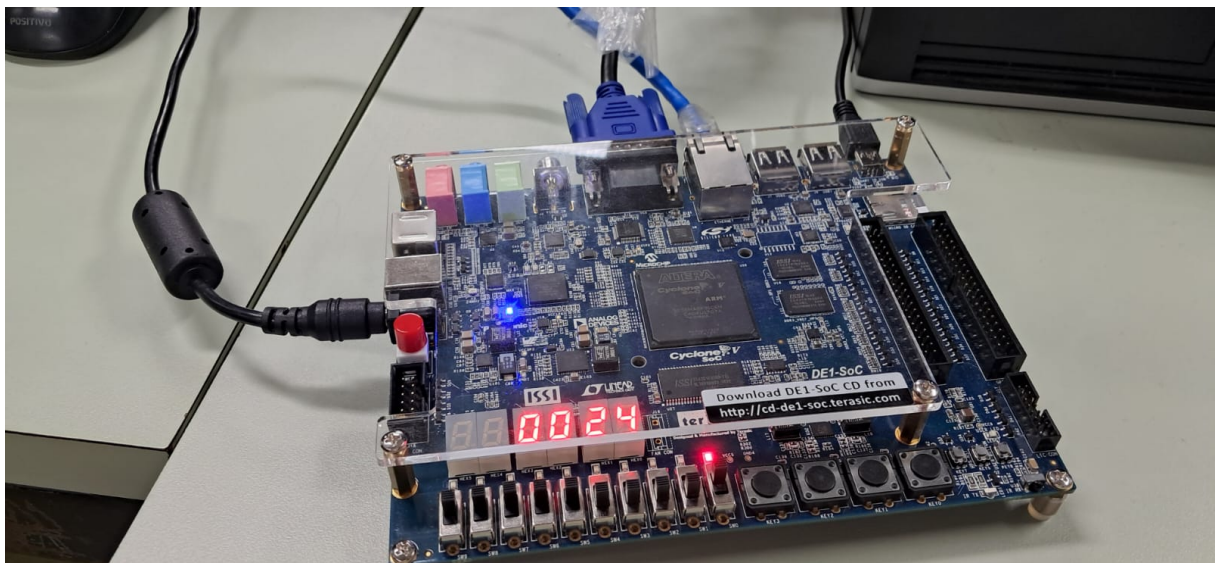


Figura 2: LEDs e display de 7 segmentos mostrando vidas/pontos

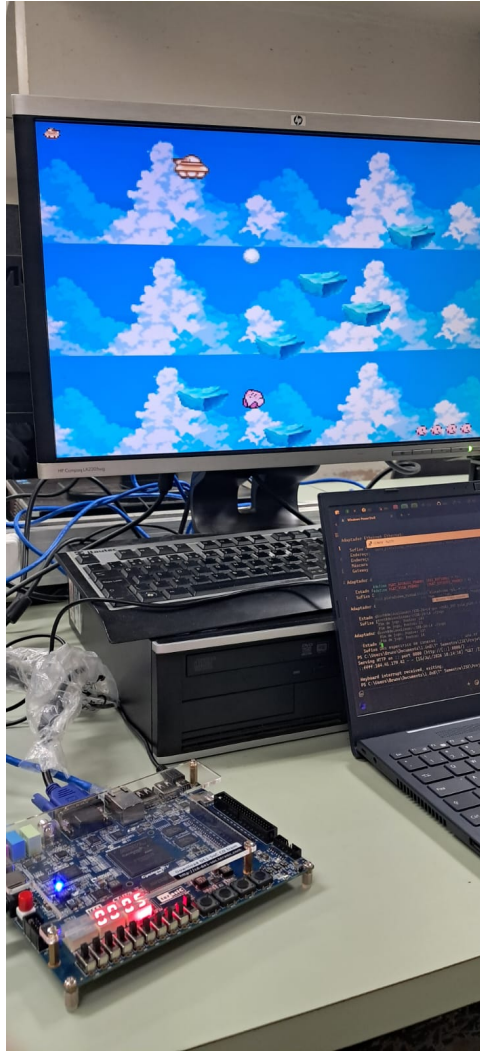


Figura 3: Partida em andamento

3 Funcionamento do jogo

O jogo segue a estrutura clássica de laço de jogo (*game loop*), implementada na função `main`, com uma máquina de estados simples por cima controlando qual tela é mostrada:

Listing 1: Laço principal, em `pula_plat.c`

```
1 while (programa_rodando) {
2     ler_entrada(&esquerda, &direita, &pular, &parry);
3     switch (estado_jogo) {
4         case ESTADO_INICIO:     renderiza_tela_inicio();    break;
5         case ESTADO_JOGANDO:    atualiza_estado(...); atualiza_vilao();
6                                 atualiza_projetil(...); atualiza_derretimento
7                                 ();
8                                 renderiza_cena(...);        break;
9         case ESTADO_GAME_OVER:  renderiza_tela_game_over(); break;
10        case ESTADO_VITORIA:    renderiza_cena(...);        break;
11    }
12 }
```


A Figura 4 mostra essa máquina de estados. O diagrama de estados interno do vilão é discutido junto com a Seção 5.

- **Tela de início:** mostra uma animação simples do personagem; apertar o botão de mover para a direita começa uma nova partida.
- **Jogando:** o núcleo do jogo — física de pulo, geração de plataformas, vilão e projétil, pontuação.
- **Game over:** mostrado quando as vidas do personagem chegam a zero; permite reiniciar ou sair.
- **Vitória:** mostrado ao alcançar a pontuação máxima; congela a cena com o personagem em pose de campeão.

Mecânicas principais:

- **Física do pulo:** gravidade e velocidade em ponto fixo 24.8 (1 pixel = 256), atualizadas a cada quadro. A colisão com uma plataforma só é testada durante a descida do pulo, fazendo o personagem pousar exatamente sobre ela.
- **Geração procedural de plataformas:** a cada plataforma nova, sorteia-se uma altura e um deslocamento horizontal compatíveis com o alcance físico do pulo (função `alcance_horizontal()`), garantindo que todo salto gerado seja sempre alcançável. Um conjunto fixo de 6 plataformas é reaproveitado indefinidamente: uma plataforma só é reciclada (reposicionada acima) depois que o jogador realmente pousou em pelo menos 3 plataformas mais recentes que ela, nunca enquanto ainda pode ser necessária como apoio.
- **Derretimento:** toda plataforma dura 12 segundos desde que é gerada, percorrendo 4 estágios visuais (gelo íntegro e 3 estágios de derretimento cada vez menores) antes de sumir e ser reciclada.
- **Câmera:** a câmera só sobe (o jogador nunca desce de tela), empurrando o cenário para baixo conforme o personagem sobe além de um limiar próximo ao topo da tela.
- **Vilão e projétil:** uma nave patrulha o topo da tela e atira periodicamente um projétil mirado na posição do personagem. Segurando o botão de *parry* no instante do impacto, o projétil é refletido de volta (reto ou na diagonal, conforme a direção escolhida) e pode acertar o próprio vilão, tirando uma de suas vidas.
- **Vidas e renascimento:** levar um tiro ou cair da tela custa uma vida; o personagem sempre renasce em cima de uma plataforma já gerada (nunca em um ponto arbitrário sem chão embaixo).

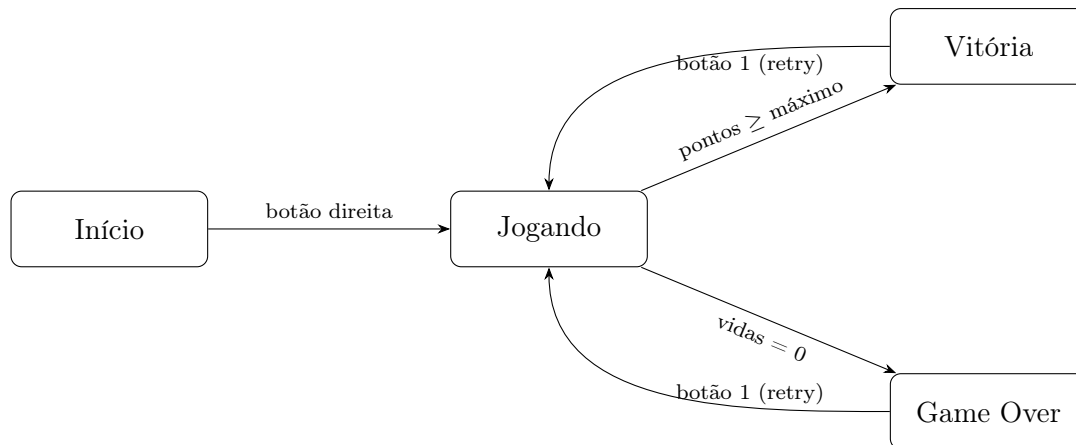


Figura 4: Máquina de estados do jogo (EstadoJogo).

4 Entradas e saídas

Periférico	Uso no jogo	Status
Vídeo VGA (obrigatório)	Renderização do cenário, plataformas e personagem	Implementado
Displays de 7 segmentos (obrigatório)	Exibição da pontuação	Implementado
LEDs	Exibição do número de vidas restantes	Implementado
Chaves (SW)	Chave 0 = pulo	Implementado
Botões (KEY)	Botões 0/1 = movimento horizontal; botão 2 = <i>parry</i> (reflete o projétil); também usados nas telas de início/ <i>game over</i>	Implementado

Tabela 1: Periféricos de entrada/saída da DE1-SoC utilizados pelo jogo.

No modo SDL_SIM (usado para desenvolvimento no PC), o teclado substitui chaves e botões:

Entrada real (DE1-SoC)	Tecla equivalente (SDL_SIM)
Botão 0 / Botão 1 (mover)	Setas esquerda/direita ou A/D
Chave 0 (pular)	Espaço ou seta para cima
Botão 2 (<i>parry</i>)	Shift

Tabela 2: Mapeamento de teclado no modo SDL_SIM.

Saídas do jogo:

- **VGA:** cenário (nuvens de fundo, que mudam de nível a cada 25 plataformas geradas), plataformas, personagem, vilão, projétil e telas de início/*game over*, todos desenhados via *double buffering* (Seção 6).
- **Displays de 7 segmentos:** pontuação atual, em 4 dígitos.
- **LEDs:** uma quantidade de LEDs acesos igual ao número de vidas restantes do personagem.

5 Estrutura do código (UML)

O jogo é implementado em C, sem orientação a objetos; ainda assim, os principais dados do jogo são organizados em **structs** (**Player**, **Plataforma**, **Vilao**, **Projetil**, **Recordes**), cada uma associada a um grupo de funções que operam sobre ela — o mesmo papel que atributos e métodos cumprem em uma classe. A Figura 5 apresenta essa estrutura como um diagrama de classes simplificado, seguindo a notação de Wolf (retângulo dividido em nome / atributos / operações).

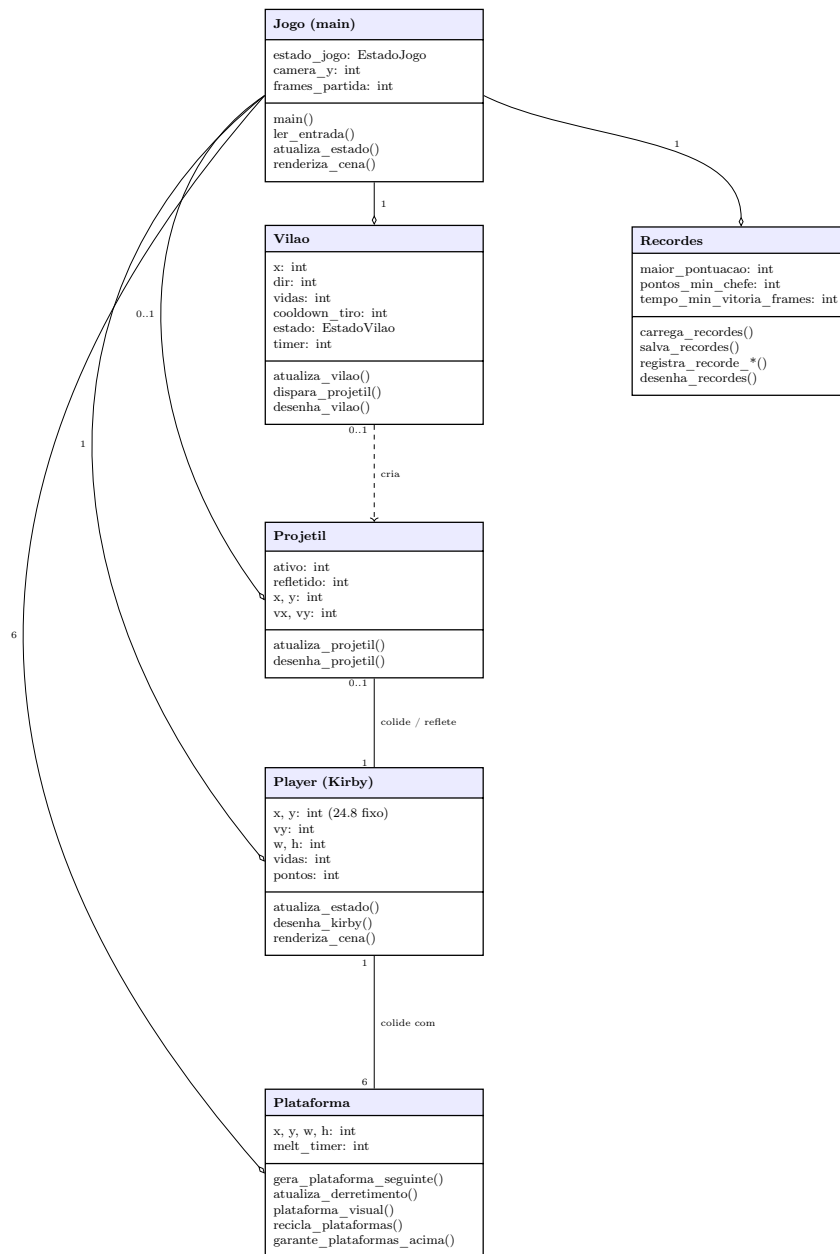


Figura 5: Diagrama de classes simplificado. Losangos indicam agregação (o objeto “todo” controla o ciclo de vida das partes); as demais linhas são associações. Jogo representa o laço principal em `main()` e as funções de nível global do arquivo.

5.1 Diagrama de estados do vilão

Além da máquina de estados do jogo (Figura 4), o vilão (**Vilao**) tem sua própria máquina de estados interna, mostrada na Figura 6: ele alterna entre patrulhar/atirar (**NORMAL**), ficar atordoado por 2 segundos após ser atingido (**ATINGIDO**), explodir por mais 2 segundos quando perde a última vida (**EXPLODINDO**) e finalmente desaparecer pelo resto da partida (**MORTO**).

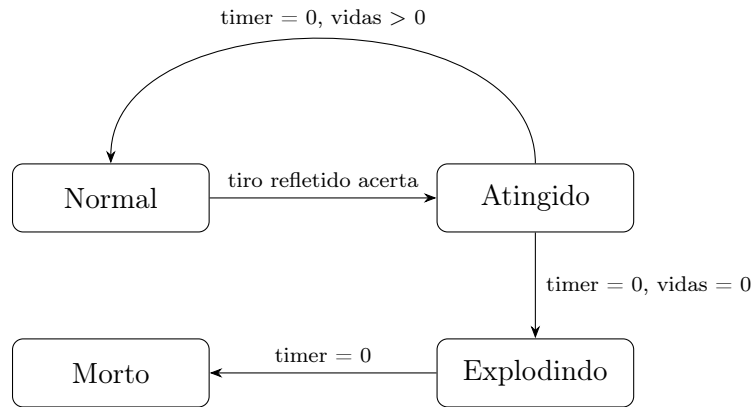


Figura 6: Máquina de estados do vilão (**EstadoVilao**).

5.2 Diagrama de sequência: parry e reflexo do projétil

A Figura 7 ilustra a interação mais original do jogo: o jogador reflete de volta o projétil do vilão segurando o *parry* no instante certo, e o projétil refletido pode, por sua vez, acertar o vilão.

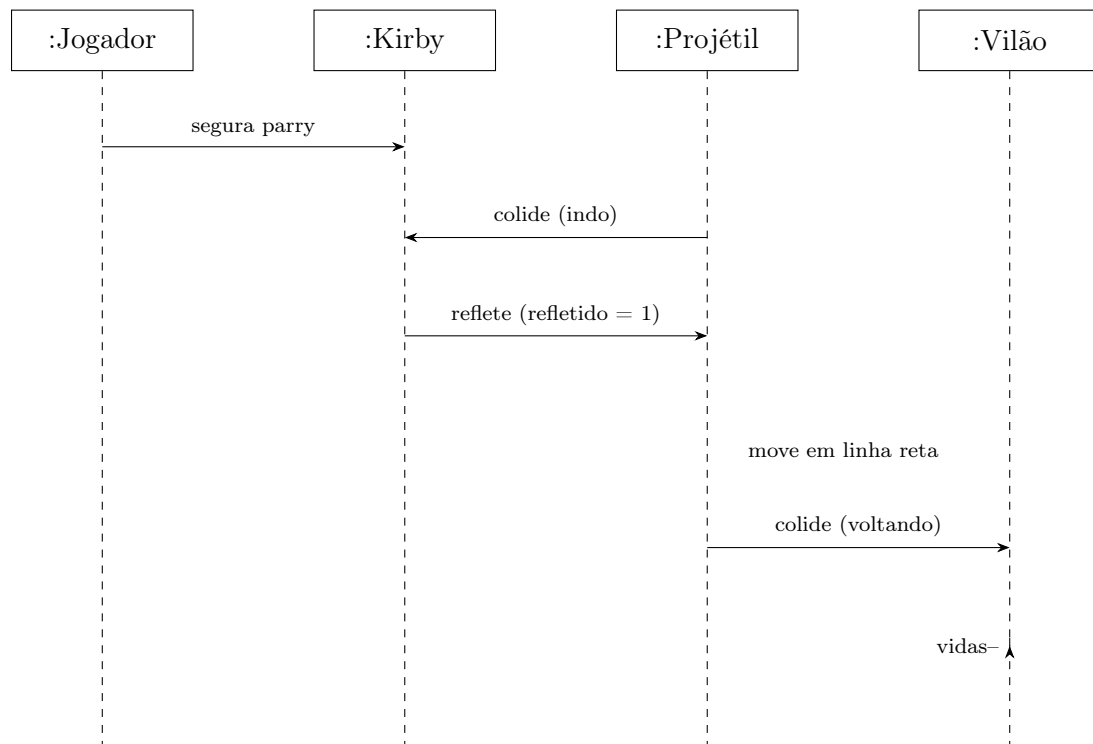


Figura 7: Diagrama de sequência simplificado da mecânica de *parry* (função `atualiza_projetil()`).

5.3 Principais módulos do arquivo

- **Física e colisão** (`atualiza_estado`, `alcance_horizontal`): gravidade, pulo, colisão com plataformas, pontuação, renascimento.
- **Geração de plataformas** (`gera_plataforma_seguinte`, `sorteia_dy`, `recicla_plataformas`, `garante_plataformas_acima`): geração procedural compatível com o alcance do pulo, reciclagem contínua do conjunto fixo de plataformas.
- **Derretimento** (`atualiza_derretimento`, `plataforma_visual`): dificuldade extra, cada plataforma tem vida útil limitada.
- **Vilão e projétil** (`atualiza_vilao`, `dispara_projetil`, `atualiza_projetil`): inimigo ativo que persegue via projéteis mirados, com mecânica de *parry*.
- **Renderização** (`desenha_sprite`, `desenha_fundo`, `renderiza_cena`, `atualiza_tela`): *double buffering* e composição da cena (Seção 6).
- **Recordes** (`carrega_recordes`, `salva_recordes`, `registra_recorde_*`, `desenha_recordes`): persistência e exibição dos recordes (Seção 7).
- **Telas** (`renderiza_tela_inicio`, `renderiza_tela_game_over`): telas de início e fim de jogo.

6 Gráficos

6.1 Framebuffer e *double buffering*

A tela é uma matriz de 320×240 pixels em RGB565 (5 bits de vermelho, 6 de verde, 5 de azul, empacotados em 16 bits). Em vez de desenhar direto na memória de vídeo, tudo é desenhado primeiro em um *back buffer* na RAM (`back_buffer[HEIGHT][WIDTH]`); só ao final do quadro esse buffer inteiro é copiado de uma vez para a memória de vídeo real (`atualiza_tela()`). Sem isso, cada `plot_pixel` apareceria na tela imediatamente, deixando visível o desenho do fundo, das plataformas e do personagem em sequência (*flicker*).

6.2 Pipeline de sprites

Os sprites não são desenhados pixel a pixel manualmente: eles vêm de imagens PNG (baixadas de *sprite sheets* ou recortadas/editadas à mão) e são convertidos para arrays C de cores RGB565 por um script auxiliar, `tools/png2c.py`. Um pixel transparente na imagem original (por canal alfa, ou por combinar com uma cor de fundo informada via `-bg`) vira uma cor-chave (*magenta*, `0xF81F`) que a função `desenha_sprite()` sabe ignorar ao desenhar. O mesmo script também lida com redimensionamento (`-resize`, com filtro `-nearest` para não borrar pixel art) e recorte de regiões de uma *sprite sheet* maior (`-crop`).

A Figura 8 mostra os diferentes sprites do personagem usados conforme o estado do jogo (parado, pulando, atordoadado, em *parry*, campeão), e a Figura 9 mostra o vilão e as plataformas em seus diferentes estágios de derretimento.

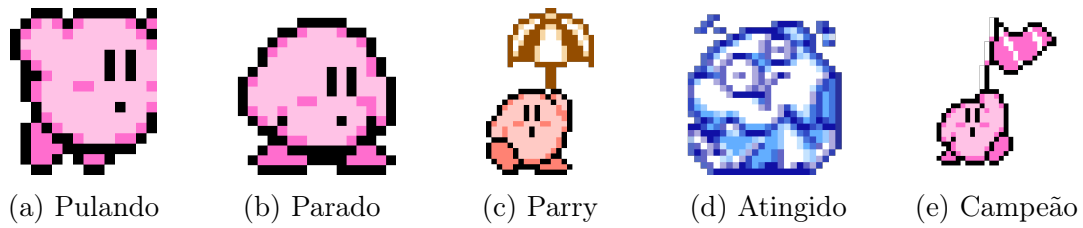


Figura 8: Sprites do personagem conforme o estado do jogo.



Figura 9: Vilão, projétil e os 4 estágios visuais de uma plataforma derretendo (cada estágio dura 3 segundos, 12 segundos no total).

6.3 Fundo e câmera

O fundo (céu com nuvens) é um único sprite maior repetido em grade por toda a tela, sem costura visível entre repetições. A fase vertical do padrão acompanha a posição da câmera, de forma que o fundo role junto com o cenário conforme o jogador sobe, em vez de ficar “grudado” na tela. A cada 25 plataformas geradas, o sprite de fundo muda para o próximo de 8 níveis disponíveis, dando a sensação de altitude crescente (do céu claro até tons mais escuros/estrelados no nível mais alto).

7 Sistema de pontuação

7.1 Pontuação

A pontuação é calculada a partir da altura recorde alcançada pelo personagem em relação à plataforma inicial (10 pixels de altura = 1 ponto), até um teto de 2000 pontos — alcançá-lo faz o jogador vencer o jogo (`ESTADO_VITORIA`). A pontuação é sempre a maior altura já alcançada na partida atual (não cai se o personagem descer ou cair), e é exibida em tempo real nos displays de 7 segmentos (ou no título da janela, no modo `SDL_SIM`).

7.2 Recordes persistentes

Além da pontuação da partida atual, o jogo mantém três recordes pessoais, salvos em um arquivo simples (`recordes.dat`) ao lado do executável, e recarregados a cada execução:

1. **Maior pontuação já alcançada**, atualizado sempre que uma partida termina (por queda, por tiro, ou por vitória).

2. **Menor pontuação em que o vilão já foi derrotado**, atualizado no instante em que ele explode.
3. **Menor tempo até vencer o jogo**, contado em quadros desde o início da partida e atualizado ao alcançar a pontuação máxima.

Esse mecanismo funciona tanto no modo `SDL_SIM` quanto na DE1-SoC real (ambos têm sistema de arquivos); no CPULATOR, sem disco persistente, a leitura do arquivo simplesmente falha silenciosamente e o jogo segue sem recordes salvos entre execuções.

Como o jogo não possui um sistema de texto livre na tela (apenas os dígitos de 7 segmentos, que não são desenhados na VGA), os recordes são exibidos de duas formas: no título da janela (modo `SDL_SIM`), e desenhados diretamente na tela nas telas de *game over* e de vitória, usando uma fonte de dígitos 3×5 pixels desenhada “na mão” com retângulos (função `desenha_numero()`), acompanhada de um ícone reduzido indicando a que recorde cada número se refere (vilão, Kirby campeão, ou Kirby neutro para o recorde geral de pontuação).

8 Dificuldades e desafios

Diversos problemas não triviais surgiram durante o desenvolvimento e a depuração do jogo, especialmente na geração procedural de plataformas — uma das partes mais originais e também mais difíceis de acertar do projeto.

- **Geração de plataformas incompatível com o alcance do pulo**: nas primeiras versões, era possível o jogador chegar à última plataforma gerada sem que nenhuma nova aparecesse à frente, porque a geração antecipada só reagia quando o personagem já estava muito perto do topo da tela. A solução foi antecipar bem mais esse gatilho (assim que o personagem cruza o primeiro terço da tela de baixo para cima, e não só o último terço perto do topo).
- **Plataformas desaparecendo debaixo do personagem**: o mecanismo que reaproveita plataformas antigas (para gerar novas acima) usava, a princípio, a posição *instantânea* do personagem para decidir quais plataformas já haviam sido “ultrapassadas” e podiam ser recicladas. Como o alcance vertical de um pulo (cerca de 46 pixels) pode superar o vão entre plataformas consecutivas (15 a 40 pixels), o personagem passa momentaneamente mais alto que a *próxima* plataforma no ápice do pulo, sem ter pousado nela — fazendo o jogo reciclar (e sumir com) uma plataforma que ele ainda precisava usar. A correção foi passar a rastrear a última plataforma em que o personagem *realmente pousou* (evento de colisão), em vez da posição em tempo real, e só reciclar plataformas comprovadamente já ultrapassadas.
- **Renascimento no vazio**: ao perder uma vida, o personagem originalmente reaparecia em um ponto fixo da tela (relativo à câmera), sem garantia de que houvesse uma plataforma exatamente ali — podendo cair de novo imediatamente, ou (pior) confundir a lógica de reciclagem, fazendo sumir as próprias plataformas que ele precisaria para se salvar. A solução foi sempre escolher, entre as plataformas já geradas, a mais próxima da altura de referência da câmera, e posicionar o personagem exatamente em cima dela.

- **Transparência perdida ao recortar sprites em editor de imagem:** ao recortar/editar alguns sprites manualmente (para diminuir a área de contorno vazia ao redor do personagem), os PNGs exportados passaram a ter o fundo “achatado” como um xadrez cinza sólido (as cores de transparência usadas visualmente por editores como Photoshop/GIMP), em vez de canal alfa real — o que fazia o xadrez aparecer de verdade no jogo em vez de transparência. A causa foi identificada inspecionando o canal alfa dos PNGs exportados (sempre opaco), e a correção foi tratar as duas cores exatas do xadrez como cor de fundo no conversor `png2c.py` (que já suportava múltiplas cores de fundo via `-bg`), sem precisar reexportar as imagens.
- **Compilação silenciosamente falhando no MSYS2:** em um ambiente de execução automatizado, invocar o `gcc` do MinGW-w64 por caminho completo falhava sem nenhuma mensagem de erro, porque o subprocesso interno do compilador (`cc1.exe`) não encontrava suas próprias DLLs de dependência (como `libmpfr-6.dll`) — o diretório real do MSYS2 (`mingw64/bin`) não estava no `PATH` do ambiente usado, que continha uma entrada equivalente, mas inválida nesse contexto. A correção foi simplesmente garantir que `mingw64/bin` estivesse no `PATH` antes de compilar.
- **Balanceamento entre física do pulo e espaçamento das plataformas:** a distância mínima e máxima entre plataformas consecutivas precisa ficar sempre dentro do alcance físico do pulo (calculado a partir da gravidade e da velocidade inicial do pulo, com uma margem de segurança), senão o jogo geraria saltos impossíveis. Esse cálculo é feito em `alcance_horizontal()`, resolvendo a equação de queda livre para o instante em que o personagem cruzaria uma dada altura na descida do pulo.

9 Conclusão

O jogo implementa um núcleo completo de *platformer* com física de pulo, colisão, inimigo ativo, obstáculo destrutível e nocivo, geração procedural infinita de cenário, sprites animados, *double buffering*, e um sistema de recordes persistentes entre execuções. O maior aprendizado do projeto não foi tanto a implementação inicial das mecânicas, mas os ajustes finos necessários para que a geração procedural de plataformas se comportasse de forma consistente com a física do jogo em todas as situações — um problema de sistemas com estado e temporização que só se revela plenamente jogando (e quebrando) o jogo repetidas vezes.